

SKILLBRIDGE BACKEND

Rapport Technique Détaillé : Architecture Modulaire & Logique Métier Spring Boot

Explication technique approfondie du code backend, du cycle de vie des requêtes REST, du moteur algorithmique de recommandation, de la sécurité par filtres JWT, et des modules de liaison Big Data.

Auteur : Omar El Khali

Rôle : Développeur Backend Spring Boot & Architecte de Données

Framework : Spring Boot v4.0 (Java 21, Spring Security, JPA Hibernate)

Date de Publication : 20 mai 2026

1. Architecture Globale & Contrôleurs REST

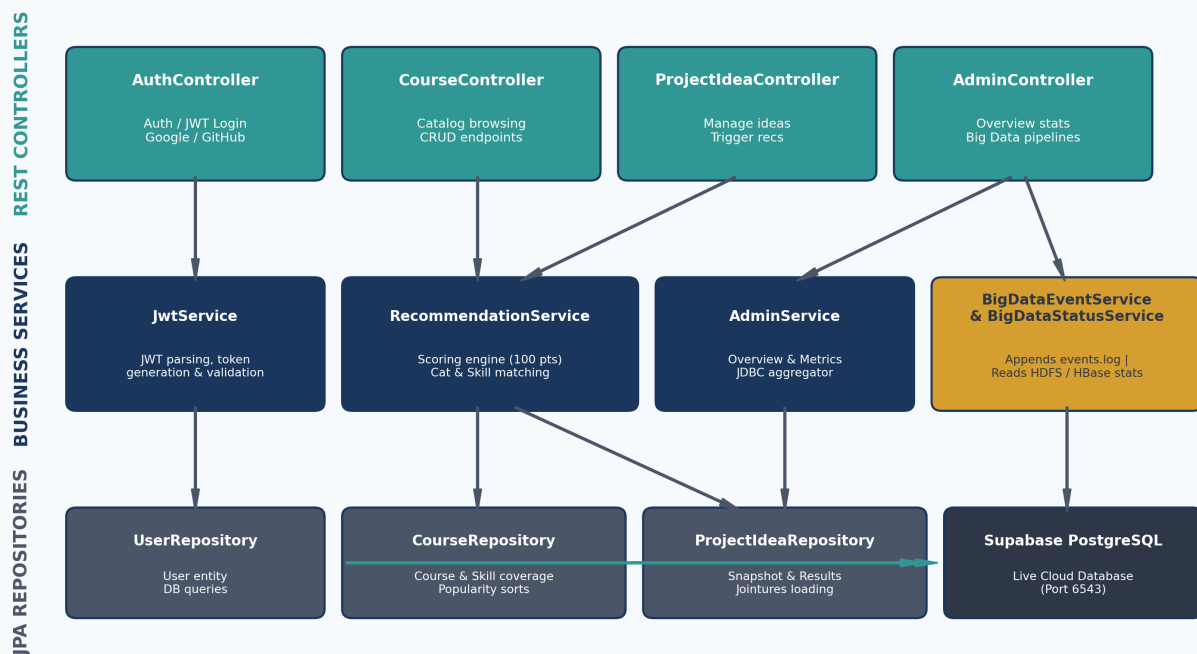
Le backend de **SkillBridge** est structuré selon un pattern d'architecture en couches standardisé (*Layered Architecture*) : **Controller -> Service -> Repository -> Database**. Chaque domaine fonctionnel (cours, compétences, projets, sécurité, analytics, big data) est isolé dans un package Java dédié. Cela garantit une maintenabilité et une testabilité optimales du code source.

Les Contrôleurs REST Clés :

Les contrôleurs Spring Boot sont annotés avec `@RestController` et exposent les endpoints JSON consommés par l'UI React :

- **AuthController** : Gère l'inscription, la connexion par mot de passe et l'échange de jetons OAuth (Google / GitHub). Endpoint: `/api/auth/*`.
- **CourseController** : Permet la recherche plein texte paginée, le filtrage par niveau ou catégorie, et le CRUD pour les administrateurs. Endpoint: `/api/courses/*`.
- **ProjectIdeaController** : Permet aux utilisateurs de gérer leurs projets et de déclencher la génération de recommandations. Endpoint: `/api/projects/*`.
- **AdminController** : Récupère les métriques consolidées de Supabase et les rapports du cluster Hadoop. Endpoint: `/api/admin/*`.

Structure des Classes du Backend Spring Boot (SkillBridge)



2. Cartographie Modulaire & Fichiers Clés du Backend

Le code source Spring Boot de **SkillBridge** est structuré par domaine fonctionnel. Chaque sous-système encapsule ses entités JPA, ses repositories d'accès aux données, ses services métier, ses DTOs d'échanges d'API et ses contrôleurs REST.

Arborescence Simplifiée des Packages (com.skillbridge) :

```
com.skillbridge
■■■■ config/          # Configurations générales (AppProperties.java, AppConfig.java)
■■■■ common/          # Exceptions globales, base de données générique
■■■■ user/            # Inscription, rôles, profils (AuthService.java, User.java)
■■■■ security/        # Chaîne de filtres JWT, validateurs Google/GitHub OAuth2
■■■■ course/          # Catalogue de cours, catégories (CourseController.java, Course.java)
■■■■ skill/           # Référentiel des compétences techniques (SkillService.java)
■■■■ projectidea/     # Soumission des idées, clichés de recommandations
■■■■ recommendation/  # Moteur de scoring temps réel (RecommendationService.java)
■■■■ bigdata/         # Ingestion events.log, communication HBase (BigDataEventService.java)
■■■■ progress/        # Suivi de la progression d'apprentissage
```

Tableau de correspondance des composants clés :

Module / Package	Composants Majeurs	Rôle Technique / Métier
security	SecurityConfig.java JwtAuthenticationFilter.java	Garantit la sécurité stateless et intercepte les requêtes pour valider le JWT.
recommendation	RecommendationService.java	Moteur de scoring hybride de 100 points reliant projets et cours.
bigdata	BigDataEventService.java BigDataStatusService.java	Ingère asynchronement les clics/recherches via un verrou ReentrantLock.
user	AuthController.java AuthService.java User.java	Gère les comptes utilisateurs, l'authentification et les privilèges d'accès.

3. Filtres de Sécurité JWT & Authentifications Sociales

La sécurité du backend est configurée via **Spring Security**. Toutes les routes API (sauf l'authentification) nécessitent un jeton JWT valide pour être accédées.

Filtre JWT et Session Stateless :

L'application utilise une politique de session *Stateless*. Le filtre custom `JwtAuthenticationFilter` intercepte chaque requête, extrait le token JWT de l'en-tête, le valide auprès du `JwtService`, charge l'utilisateur depuis `CustomUserDetailsService` et l'injecte dans le contexte de sécurité de Spring.

Authentifications tierces (Google & GitHub) :

Pour offrir une expérience fluide, SkillBridge prend en charge la connexion sociale :

1. **Google OAuth2 (`GoogleTokenVerifierService.java`)** : Le frontend envoie le Token ID Google reçu après authentification. Le service backend valide la signature du token à l'aide des clés publiques de Google et de l'ID d'application (Client ID). Si l'utilisateur est authentique, il est inscrit ou connecté sur le champ.
2. **GitHub OAuth2 (`GithubOAuthService.java`)** : Le frontend envoie un code d'autorisation temporaire. Le service backend effectue une requête POST à GitHub pour échanger ce code contre un Access Token sécurisé, puis l'utilise pour récupérer les détails du profil utilisateur.

Validation de Jeton Google (`GoogleTokenVerifierService.java`) :

```
public GoogleIdentity verify(String idToken) { Jwt jwt; try { jwt = jwtDecoder.decode(idToken); //
Décodage et validation signature / audience } catch (RuntimeException ex) { throw new
BadRequestException("Invalid Google token."); } String email = stringClaim(jwt, "email"); Boolean
emailVerified = jwt.getClaim("email_verified"); if (email == null || email.isBlank() ||
!Boolean.TRUE.equals(emailVerified)) { throw new BadRequestException("Google account email must be
verified."); } return new GoogleIdentity( email.trim().toLowerCase(Locale.ROOT), stringClaim(jwt,
"given_name"), stringClaim(jwt, "family_name") ); }
```

Configuration de la chaîne de filtres Spring Security (`SecurityConfig.java`) :

```
@Bean public SecurityFilterChain filterChain(HttpSecurity http) throws Exception { http.csrf(csrf ->
csrf.disable()) .sessionManagement(sess ->
sess.sessionCreationPolicy(SessionCreationPolicy.STATELESS)) .authorizeHttpRequests(auth -> auth
.requestMatchers("/api/auth/**").permitAll() .requestMatchers("/api/admin/**").hasRole("ADMIN")
.anyRequest().authenticated() ) .addFilterBefore(jwtFilter,
UsernamePasswordAuthenticationFilter.class); return http.build(); }
```

4. Le Cœur Algorithmique : Moteur de Recommandations

Le RecommendationService.java analyse les projets d'études et score les cours du catalogue sur un total de 100 points.

Algorithme de Scoring (100 Points Max) :

Chaque cours candidat du catalogue (parmi 17 075) est noté selon quatre dimensions clés :

1. **Title Match (30 points max)** : Chaque mot-clé extrait du projet qui correspond à un mot du titre du cours ajoute **6 points**. Si une compétence détectée exacte apparaît dans le titre du cours, un bonus de **8 points** est accordé.
2. **Skill Match (40 points max)** : Si le cours est explicitement lié à une compétence détectée du projet (liaison N:N), le cours gagne **10 points**. Si le nom de la compétence est simplement mentionné dans la description du cours, il gagne **4 points**.
3. **Category Match (20 points max)** : Si la catégorie principale du cours correspond à une catégorie détectée, le cours gagne **20 points**. En cas de détection indirecte par mot-clé dans les descriptions, il gagne **10 points**.
4. **HBase & Popularity Bonus (10 points max)** : Un score de popularité inhérent au catalogue (calculé à partir des plateformes Coursera/edX) offre jusqu'à **8 points** ($\text{popularity_score} / 10$). Si la base NoSQL HBase indique que le cours a déjà fait l'objet d'activités (clics, progression, sauvegardes), un bonus analytique de **2 points** est ajouté.

Calcul de scoring dans RecommendationService.java :

```
int titleScore = Math.min(30, titleKeywords.size() * 6 + matchedSkills.stream().filter(skill ->
normalizeText(course.getTitle()).contains(normalizeText(skill))).mapToInt(ignored -> 8).sum()); int
skillScore = Math.min(40, matchedSkills.size() * 10 + descriptionSkillHits.size() * 4); int
categoryScore = matchedCategories.isEmpty() ? 0 : 20; if (categoryScore == 0 &&
matchedCategoryKeywords.values().stream().flatMap(List::stream).anyMatch(kw ->
normalizeText(course.getTitle() + " " + course.getDescription()).contains(normalizeText(kw)))) {
categoryScore = 10; } int bonusScore = bonusScore(course); // Popularité (max 8) + HBase stats hit (2
pts) int totalScore = Math.min(100, titleScore + skillScore + categoryScore + bonusScore);
```

Stratégie de Fallback :

Si l'idée de projet est trop courte ou qu'aucune compétence n'est détectée directement, le moteur de recommandation démarre automatiquement une recherche par mots-clés (recherche floue ILIKE) sur les titres et descriptions, puis retombe sur les cours les plus populaires du catalogue afin de ne jamais renvoyer une page vide à l'étudiant.

5. La Liaison Big Data en Ingestion et Cache NoSQL

Le backend ne se contente pas de servir l'application web ; il fait le pont en temps réel avec le cluster Big Data.

Ingestion de Flux Événementiel (**BigDataEventService.java**) :

Lors de chaque action majeure de l'étudiant (clic, recherche, sauvegarde de cours, recommandation), le backend appelle la méthode `appendEvent()`. Pour éviter tout blocage ou corruption de fichier dans un environnement hautement concurrent, le service implémente un verrou exclusif réentrant (**ReentrantLock**). Les événements sont sérialisés au format JSON-Line dans le fichier local **events.log**, qui est ensuite surveillé par l'agent Apache Flume pour être déversé dans HDFS.

Code du service d'ingestion sécurisé :

```
public boolean appendEvent(String eventType, Map fields) { Map event = new LinkedHashMap<>();
event.put("eventType", eventType); event.put("source", "web-app"); event.put("timestamp",
Instant.now().toString()); event.putAll(fields); writeLock.lock(); // Garantit l'absence de
corruption de events.log try { Path path = eventLogPath(); Files.createDirectories(path.getParent());
Files.writeString(path, toJsonLine(event), StandardCharsets.UTF_8, StandardOpenOption.CREATE,
StandardOpenOption.APPEND); return true; } catch (IOException ex) { log.warn("Unable to append Big
Data event {}", eventType, ex); return false; } finally { writeLock.unlock(); } }
```

Lecteur de Cache Analytique HBase (**BigDataStatusService.java**) :

Pour le calcul des bonus de popularité, le backend interroge le cache analytique HBase. Puisque HBase est hébergé sur le cluster Docker et que son scan de table peut être lourd, un script analytique Python récupère périodiquement les statistiques et les compile dans un fichier résumé **bigdata-summary.json**. Le service `BigDataStatusService` charge ce résumé en mémoire et l'interroge en un temps record $O(1)$, garantissant des temps de réponse d'API inférieurs à 20ms.

6. Connexion Supabase PostgreSQL & Pooler de Connexions

La base de données principale est hébergée sur le cloud de **Supabase** (PostgreSQL 17.6). Pour des raisons de scalabilité et de robustesse, une configuration spécifique a été mise en place au niveau du pooler de connexions.

Configuration du Port 6543 (Transaction Mode) :

Par défaut, le port 5432 de Supabase fonctionne en *Session Mode*, ce qui limite le nombre total de connexions concurrentes à environ 15 sessions. Avec le trafic web et l'exécution d'API Spring Boot, cette limite peut être rapidement dépassée, provoquant l'erreur fatale FATAL: remaining connection slots are reserved for non-replication superuser connections.

Pour y remédier, le backend se connecte au **port 6543**, qui utilise le **PgBouncer Transaction Mode Pooler** de Supabase. Ce mode permet de réutiliser instantanément les connexions à chaque transaction de courte durée sans garder la connexion active pendant toute la session utilisateur, démultipliant ainsi le nombre d'utilisateurs simultanés.

Optimisation du Pool Hikari (application.properties) :

Le gestionnaire de connexions par défaut de Spring Boot, **HikariCP**, a été finement configuré :

- maximum-pool-size = 3 : Restreint le pool interne de l'application à 3 connexions pour ne pas saturer le pooler Supabase.
- minimum-idle = 1 : Garde au moins 1 connexion ouverte en veille pour un démarrage ultra-rapide des requêtes.
- idle-timeout = 10000 : Libère rapidement les connexions inactives pour éviter l'épuisement.

Conclusion sur la Robustesse du Backend

Le backend de SkillBridge est un exemple d'ingénierie logicielle Spring Boot combinant :

1. Une ****Sécurité Robuste**** avec filtrage JWT stateless et authentications sociales tierces.
2. Un ****Moteur de Recommandation**** puissant avec double matching (compétences N:N et recherche textuelle plein texte).
3. Une ****Liaison Big Data**** fluide et asynchrone en temps réel via un fichier tampon à verrou réentrant.
4. Une ****Stabilité Exceptionnelle**** de base de données grâce au port transactionnel Supabase et un pool Hikari dimensionné à la perfection.

Cette rigueur technique assure des performances de classe entreprise et une résilience complète sous charge.